

Autonomous Systems

Reinforcement Learning for Decision-Making

Group 18

Final Presentation

David Gaspar - 106541

Francisco Valério - 106533

Miguel Carneiro - 106218

Pedro Mónico - 106626

Theoretical Fundamentals

- Markov Decision Processes
- Solving the MDP
- Model Free Setting
- Q-Learning Algorithm

Robot Hardware & Software

- Alphabot2 Components
- Obstacle Sensors
- Line Sensors

Robot Control

- Control Methodology
- PD Controller
- Junction Control
- Progress Overtime

Python Micro-Simulator

- Control Methodology
- PD Controller
- Junction Control
- Progress Overtime

Experimental Validation

- Real Environment
- Robot Calibration
- Modes of Operation
- Results

References

Theoretical Fundamentals

A **Markov Decision Process (MDP)** is a formal mathematical model for decision making under uncertainty, where outcomes are partly random and partly controlled by the agent.

It is defined by

$S \rightarrow$ Set of possible **states** the robot can be in (position, angular orientation, velocity, acceleration)

$A \rightarrow$ Set of **actions** available to the robot (forward, backwards, right, left)

$T(s, a, s') \rightarrow$ **Transition probability** of reaching s' from s after action a

$R(s, a) \rightarrow$ **Reward** received for taking action a in state s

$\gamma \in [0,1] \rightarrow$ **Discount factor**, balancing immediate ($\gamma = 0$) vs future ($\gamma = 1$) rewards

The future depends only on the **current state and action** $\rightarrow p(s_{t+1} \mid s_t, a_t)$

A **policy** π maps states to actions telling the robot what is **the most optimal to do in every single situation**.

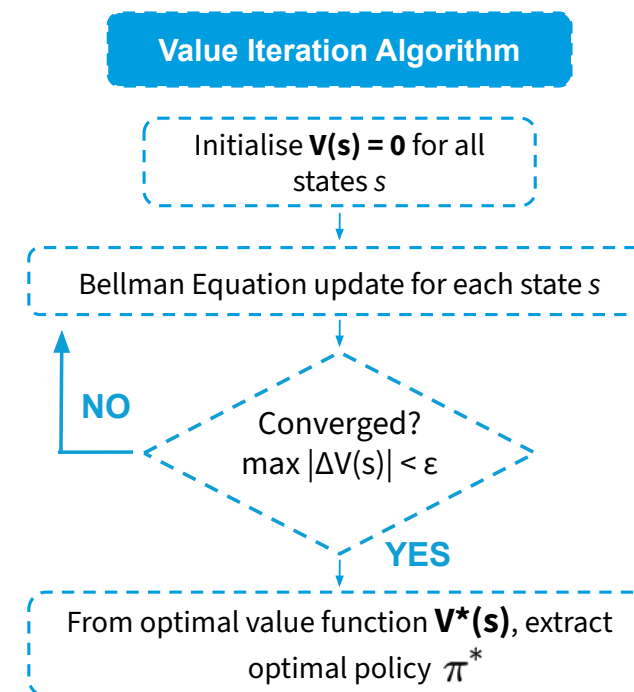
To find **the optimal policy** π^* , we first compute iteratively the value function $\mathbf{V}^*(\mathbf{s})$ (a measure of how good it is to be in state s assuming optimal behavior from that point onward) which is defined recursively by the **Bellman Optimality Equation**:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') V^*(s') \right] \quad [1]$$

The Bellman Optimality Equation is solved by iterative methods, such as **Value Iteration**:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + \gamma V_k(s')] \quad [2]$$

converges to the optimal value function $\mathbf{V}^*(\mathbf{s})$



An agent learns to make decisions by interacting with an environment since it **has not prior knowledge** of it.

Unlike in MDPs, the policy must be **learned from experience** rather than computed from a **known model**.

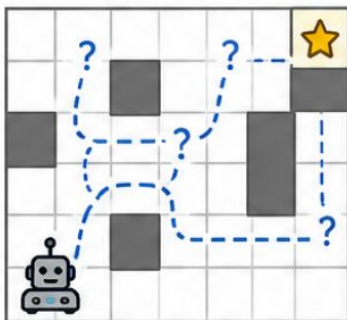
Reinforcement Learning →

When **transition model** $T(s, a, s')$ is **unknown**

Robot learns optimal behavior through **trial and error**

Action selection is driven by Exploration & Exploitation trade-off

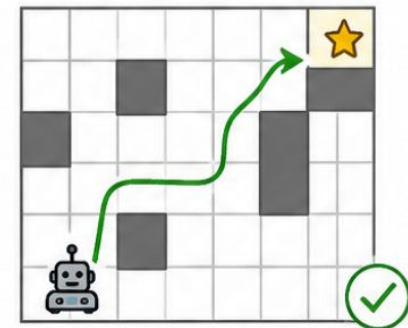
EXPLORATION
(Try new actions)



Try new actions to discover better strategies

Use current best-known actions to maximize reward

EXPLOITATION
(Use learned knowledge)



Q-Learning Algorithm →

Learns action-value function $Q(s,a)$ **directly from experience**

Updates Q-values based on **observed rewards and transitions**

Converges to **optimal policy Q^*** given sufficient exploration

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad [3]$$

$Q(s, a)$
Current value

α
Learning rate
($0 < \alpha \leq 1$)

r
Reward received

γ
Discount factor
($0 \leq \gamma < 1$)

$\max_{a'} Q(s', a')$
Best future value

$Q(s', a')$
Value of next state-action

Q-Learning Algorithm

Observe current state s

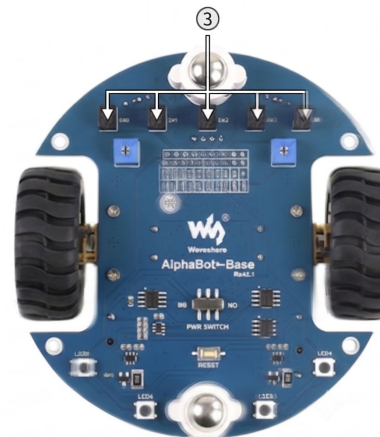
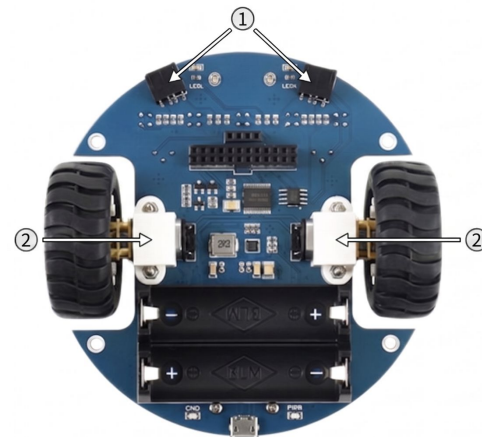
Action Selection ϵ -greedy (Explore vs Exploit)

Execute action a

Receive reward r and next state s'

Update $Q(s, a)$

Robot Hardware & Software



AlphaBot2 Hardware [5]

Main Components

Raspberry Pi 3 B

RPi Camera (B)

ST188 Sensor (2x)

N20 micro gear motor (x2)

ITR20001/T Sensor (5x)

Each **ST188 sensor** emits infrared light forward and detects how much is reflected back from nearby objects to determine whether an obstacle is present.

→ **Distant Obstacle** → low reflection → low value

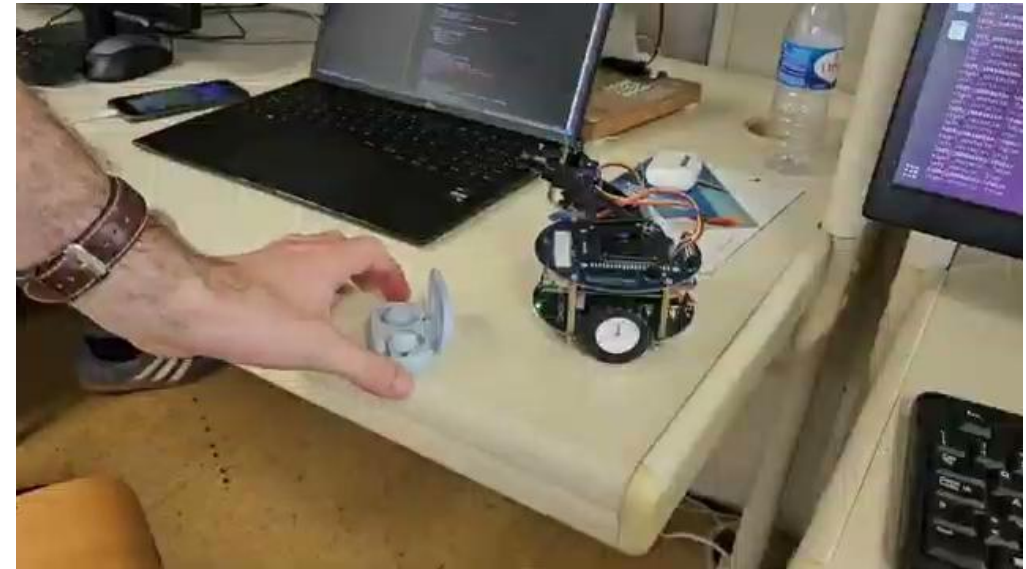
→ **Close Obstacle** → high reflection → high value

ROS2 Topic (ir_obstacles_sensors)
publishes a **Boolean Output** [4]

right_obstacle = true/false

left_obstacle = true/false

Black obstacles could not be used during
Dynamic Mode due to the low reflection



Obstacles Sensors Experiments

Each sensor ITR20001/T emits Infrared Light downward and reads how much is reflected back

Dark surface (line) → low reflection → low value

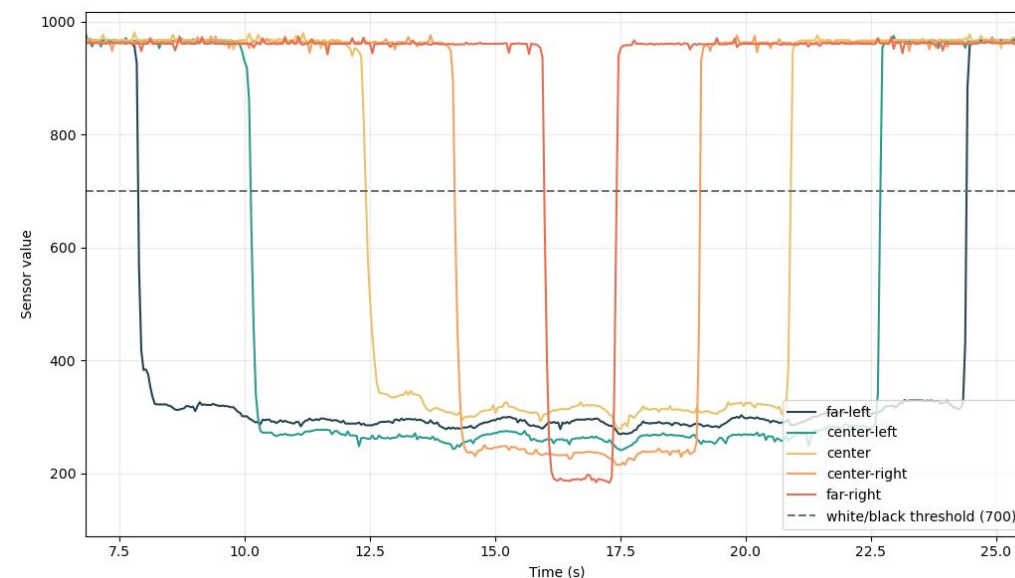
Light surface (floor) → high reflection → high value

Sensor outputs a value between 0 and 1023

~300 → sensor is directly over the line (black)

~980 → sensor is over the floor (white)

A custom node was created (**ir_line_sensors**) to publish the raw readings of each ITR20001/T through a ROS2 topic [6]



Line Sensors Experiments

Motion Control

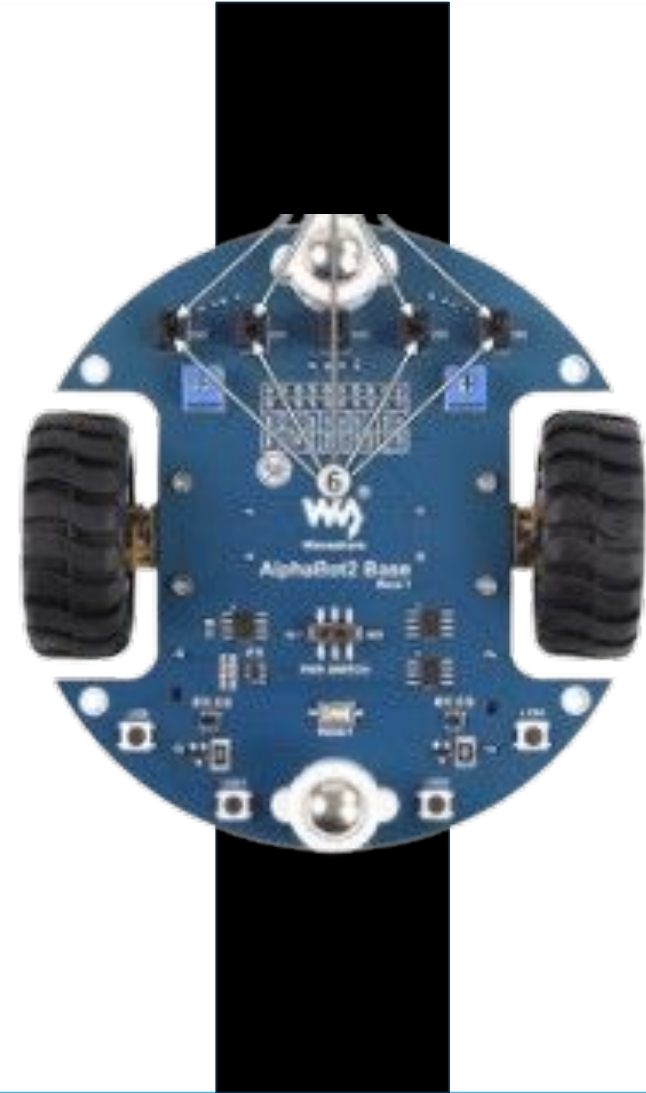
Forward Movement Control Methodology

Control is carried out using infrared (IR) sensors positioned beneath the AlphaBot2

- 01 Ideal State:** The 3 central sensors should detect “black” (line)
- 02 Deviation Detection:** If a sensor detects “white,” the robot identifies a deviation
- 03 Error Signal :** Weighted sum of the five sensor readings, representing lateral deviation from the line.

$$p_i \in \{-2, -1, 0, +1, +2\}$$

$$e(t) = \frac{\sum_{i \in \mathcal{A}} p_i}{|\mathcal{A}|}$$



PD Controller

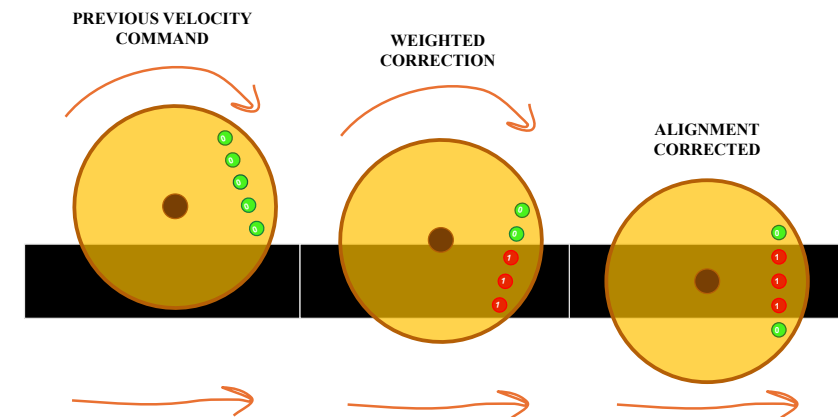
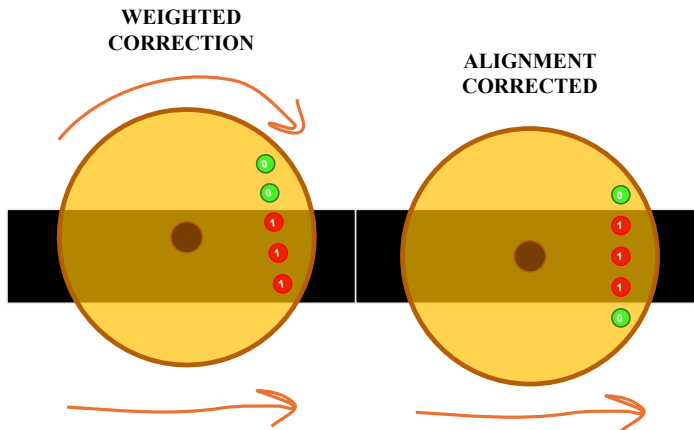
Control Law : Angular velocity correction computed, with gains tuned empirically



$$\omega(t) = K_P e(t) + K_D \Delta e(t)$$

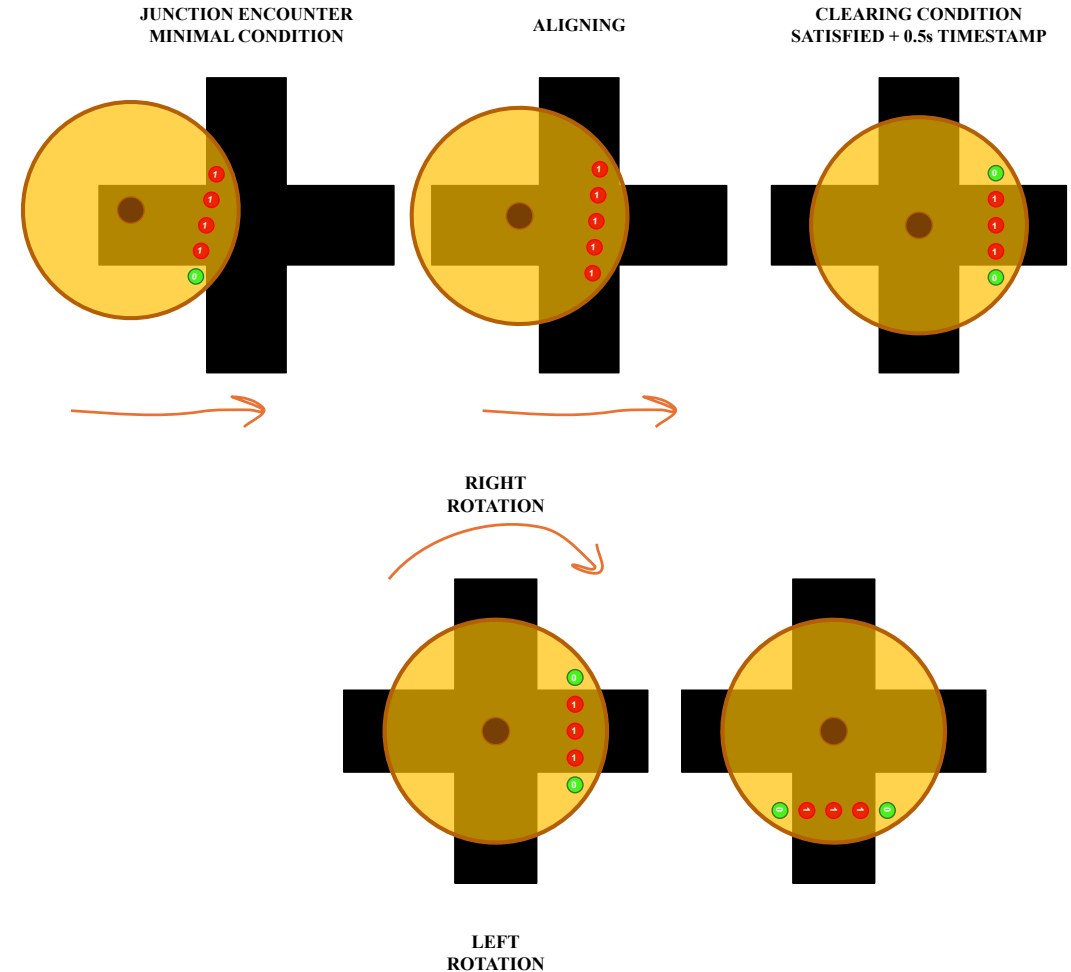
→ **Line Following** : Continuously recentres the robot over the line via differential motor speed adjustment.

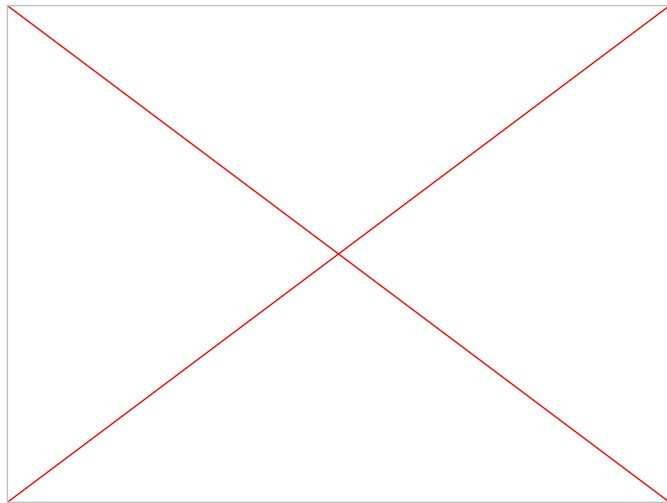
→ **Line Loss Recovery**: When no sensor detects black, the last known error drives the correction until the line is reacquired (passive recovery)



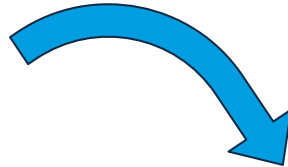
Junction Methodology

- 01 Junction Detection:** When at least 4 adjacent sensors detect "black". e.g.: $[0,1,1,1,1] \vee [1,1,1,1,0]$
- 02 Align Phase:** The robot moves forward until both outer sensors read white, or the total number of black sensors drop to three or fewer.
- 03 Search Phase:** The robot rotates until finding a new line again e.g.: $[0,1,1,1,0]$

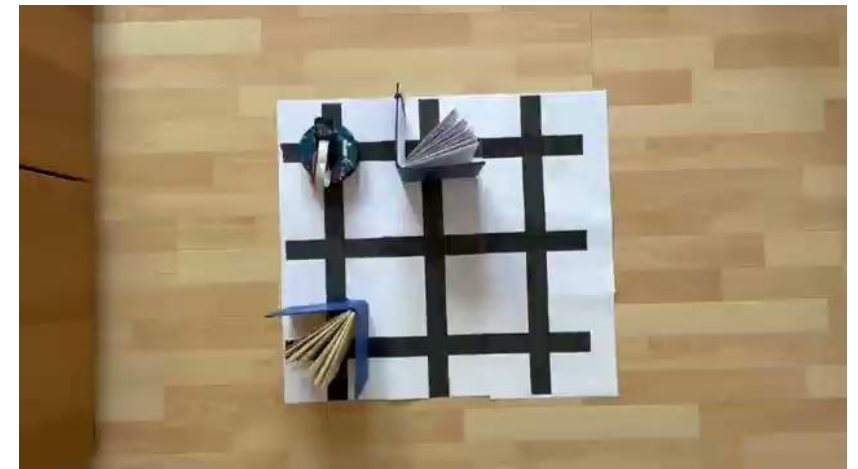
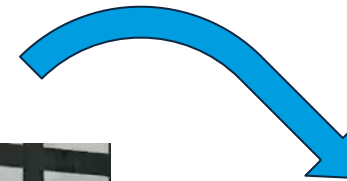


**First Motion Control Algorithm:**

Boundary conditions too strict
No Controller defined

**Latest Motion Control Algorithm:**

Cooldown timer
PD controller
Larger line width
More flexible boundary conditions



Python Micro-Simulator

Three Approaches to Navigation

Model-Based

Value iteration

- Full **environment knowledge**
- Optimal **offline** policy

Model-Free

Q-learning

- **Trial-and-error** exploration
- Builds **Q-table**
- Greedy policy rollout

Dynamic

Q-learning + Replanning

- **Unknown** obstacles
- Online **sensing**
- Value iteration conflict resolution

Objectives

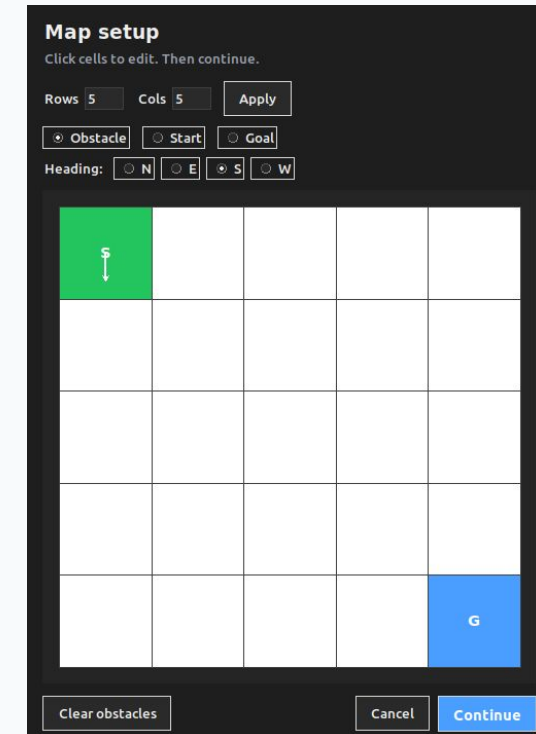
- Create a **controlled** test environment
- **Compare** navigation strategies
- **Evaluate** performance metrics

Metrics

- Path length
- Number of turns
- Convergence time
- Iterations to convergence

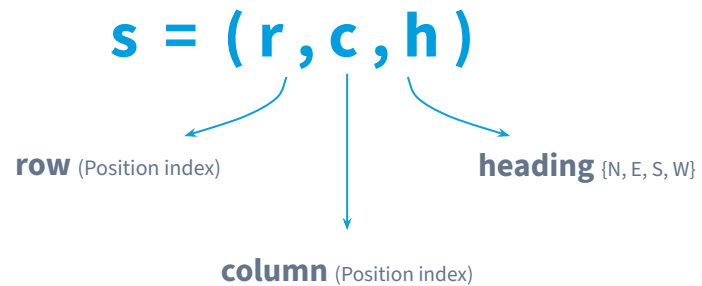
Customization

- Change the maze layout
- Add dynamic obstacles
- Modify agent positions
- Observe final policy updates



State Space and Actions

State Representation



Available Actions

Agent Control Set:

- Move Forward (**S**)
- Turn Left (**L**) 90°
- Turn Right (**R**) 90°
- Turn Around (**A**) 180°

The navigation problem is represented as a **Markov Decision** Process. Each state contains the robot **position** and **heading**

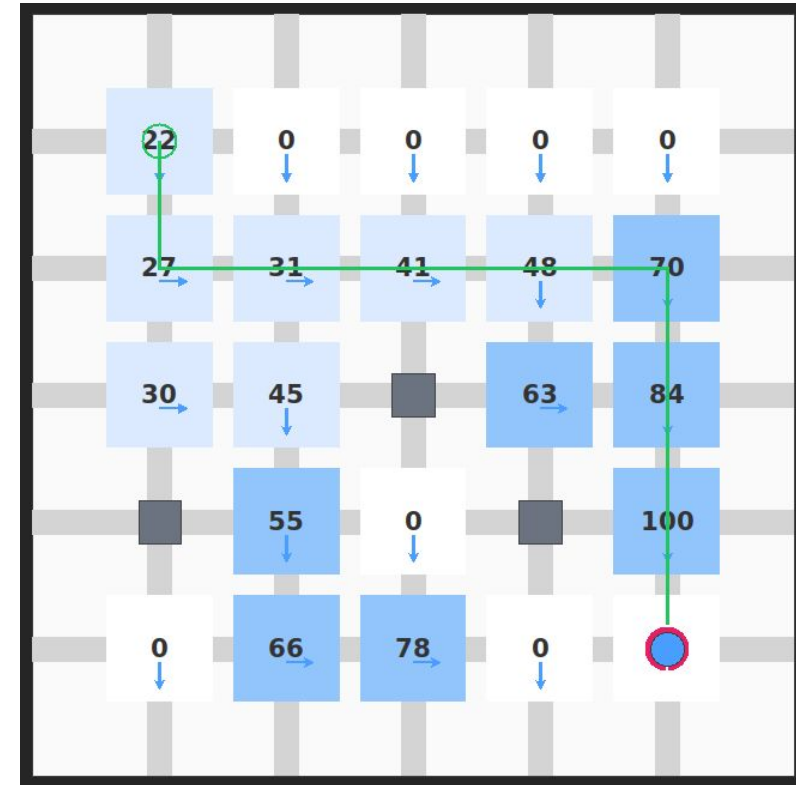
Aspect	Q-learning	Value Iteration
Output Space	$R \times C \times 4$	$R \times C \times 4$
Learns	Action-value function $Q(s,a)$	State-value function $V(s)$
Policy Extraction	$\pi^*(s) = \operatorname{argmax}_a Q(s,a)$	Derived from $V^*(s)$ using Bellman optimality
Final Output	Q-table + policy	Value function + policy

```

Heading N:
→ ↓ # → ←
↑ ↓ # → →
# ↓ # → ↓
↑ ↓ # → ↑
# ↑ ↑ ↑ ↑
Heading E:
→ ↓ # → ↓
→ ↓ # → ↓
# → # → ↓
→ ↓ # → ↓
# → → → →
Heading S:
↓ ↓ # ↓ ↓
→ ↓ # ↓ ↓
# ↓ # ↓ ↓
↓ ↓ # → ↓
# → ↓ → ↓
Heading W:
↓ ← # ↓ ←
→ ← # ← ←
# ← # ↓ ←
→ ← # → ←
# → → ← ←
  
```

Strategic Goals

- **Reach Goal**
- **Avoid** Obstacles
- **Minimize** Path Length
- **Minimize** Rotations
- **Adapt** to Environmental Changes



Overview

Characteristics

- Full environment knowledge
- Deterministic transition model
- Value Iteration
- Gauss-Seidel acceleration

Parameters

- Discount Factor (γ)
- Reward Structure

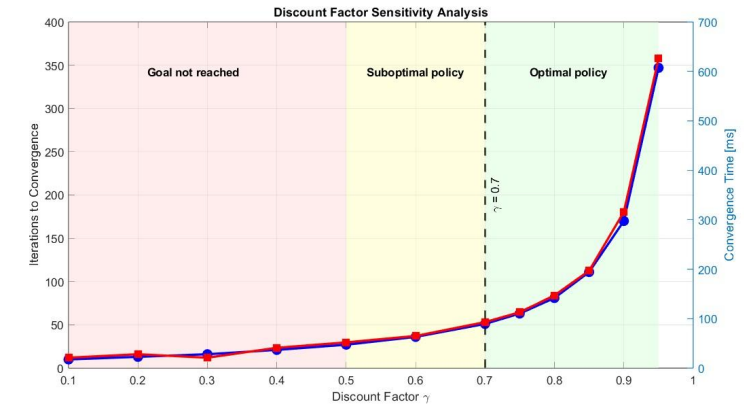
Discount Factor (γ) Impact

Low ($\gamma \leq 0.4$): Fast convergence, poor reward propagation (**no goal reaching**).

Medium ($0.5 \leq \gamma < 0.7$): Goal **reached**, but paths are **sub-optimal**.

High ($\gamma \geq 0.7$): Optimal policy achieved; **slower** convergence as $\gamma \rightarrow 1$.

Final Choice: $\gamma = 0.85$ (Reliable across grids and runs)

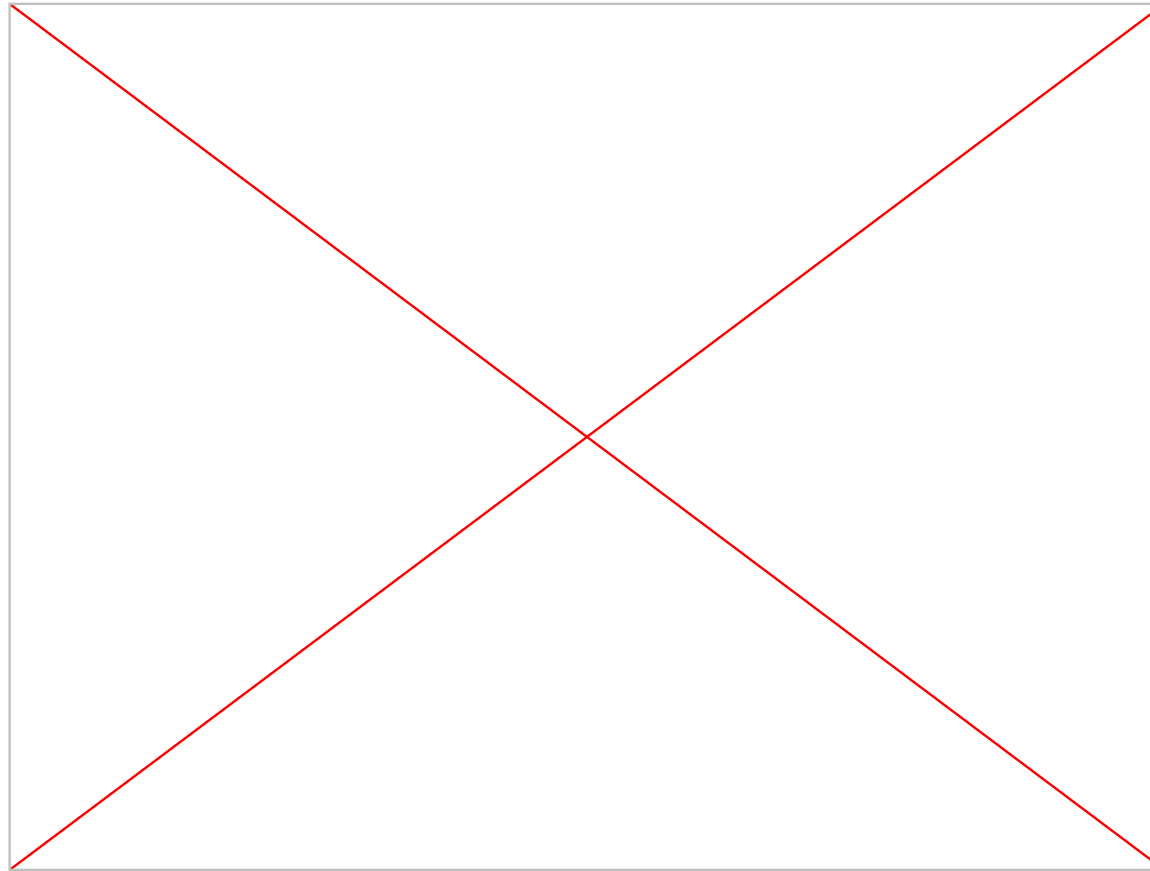


Reward Structure Insights

- Obstacle penalty must be the **most negative** to enforce valid behavior (enforces feasibility constraints).
- **Reward scaling** is critical: wrong ratios **break optimality** even after convergence
- **Turning** costs control trajectory **efficiency**
- Relative **hierarchy** is critical for stability.

Forward Step
-1
 L/R Turn
-5
 Turn-around
-10
 Obstacle
-50
 Reached Goal
+100

Micro-Simulator



Overview

Characteristics

- No prior knowledge of environment dynamics
- Learns purely from interaction (trial-and-error)
- Q-learning based policy acquisition

Parameters

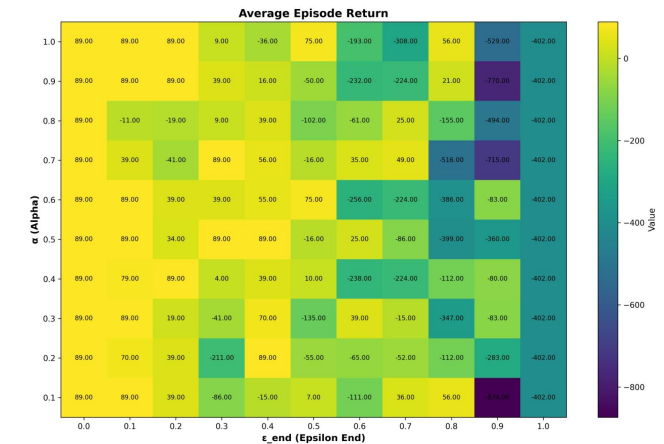
- Learning rate α
- Exploration: $\epsilon_{start} \rightarrow \epsilon_{end}$
- Discount factor γ
- Reward structure

Exploration vs Performance Trade-off

Low exploration: Highest cumulative reward, **shorter and more efficient paths.**

High exploration: Success rate maintained, but **less efficient trajectories** due to random actions.

Excessive exploration reduces policy efficiency without improving success rate.

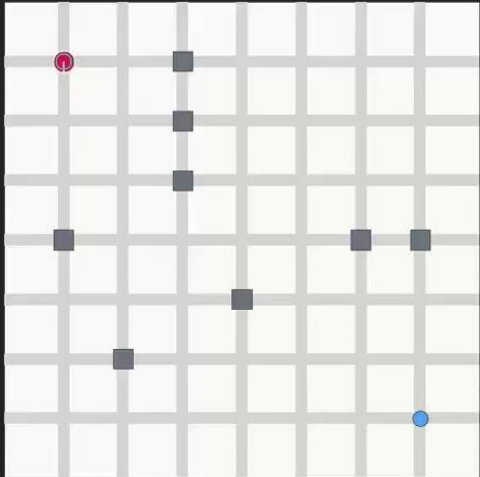


Optimal Behaviour

- Best trade-off observed at **moderate exploration levels**, balancing:
 - **Sufficient** state-space **coverage**
 - **Effective exploitation** of learned Q-values
- The learned policy achieves **high robustness**, with a success rate of ~96–98%
- Performance is **highly robust to learning rate α**
- The exploration rate (ϵ_{end}) is the dominant factor for **efficiency**

Alpha α
0.2
Exploration
 $\epsilon_{start} = 0.2$
 $\epsilon_{end} = 0.05$
decay = **0.995**
Episodes
1000
Max Steps/Episode
50

Micro-Simulator



Model-free

Episodes: 0 / not started
 Step: —
 ϵ : 1.000
 Goals reached: 0 / 0 episodes
 Press Next step or Run episode

Next step
 Run episode
Train all episodes
 Show policy (text)
 Reset Q-table

Final policy (after all episodes)

☒ N
 ☐ E
 ☒ S
 ☐ W

After training: max Q per cell + blue arrow (where to go). Green path = rollout from start. Policy on grid / text: available after all episodes finish.

Parameters

alpha 0.2
 gamma 0.85
 epsilon_start 1.0
 epsilon_end 0.05
 epsilon_decay 0.995
 max_episode_steps 500

Dynamic Micro-Simulator

(Simulates **realistic robotic navigation** in changing environments)

Two-Stage Framework

1. Offline (Model-Free)

Q-learning on initial known map to produce **baseline policy**

2. Online (Dynamic)

Execution in evolving environment with **new obstacles**

Perception & Adaptation

- **Local sensing** during execution
- Incorporates **detected obstacles**
- Continuous internal map updates
- **Replanning**: Value Iteration
- Ensures recovery from changes

Two Map Representations

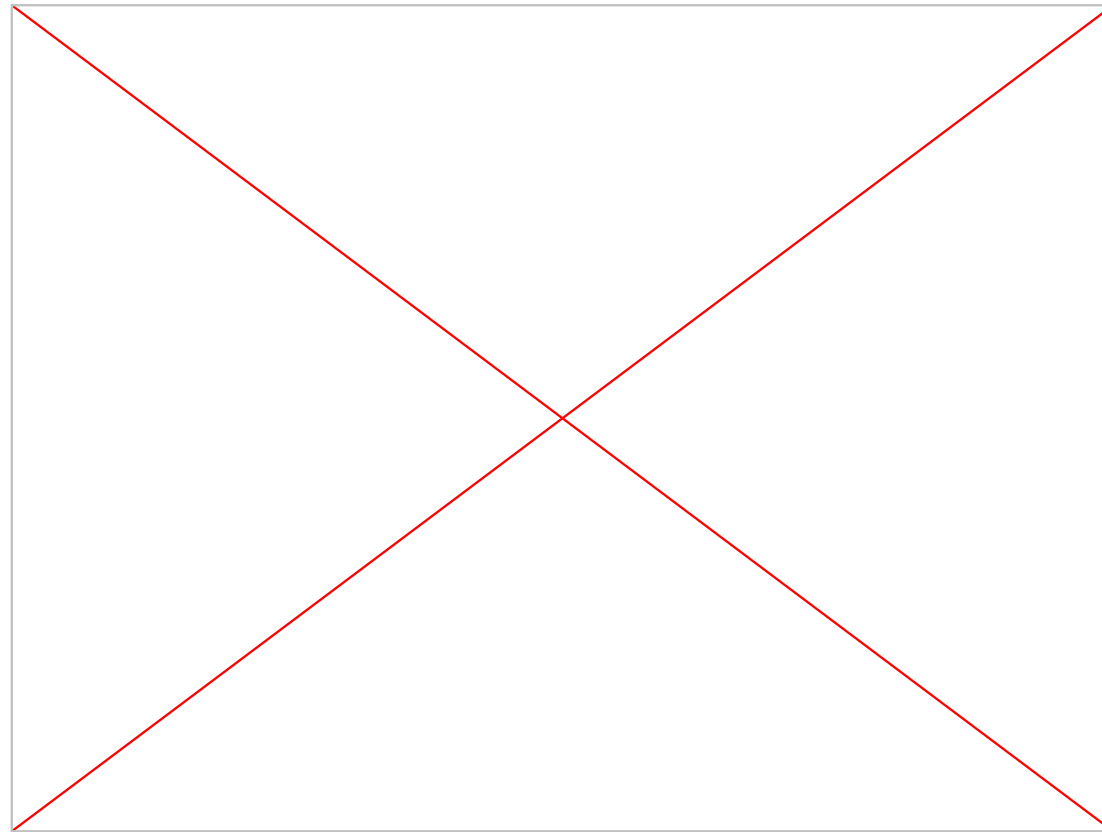
Known Map (Robot's Map)

- Agent's internal belief
- Used for planning

True Map (Ground Truth)

- Actual environment state
- Not directly accessible
- For simulation

Micro-Simulator



Experimental Validation

The real environment was composed of 5x7 maze composed of cells:

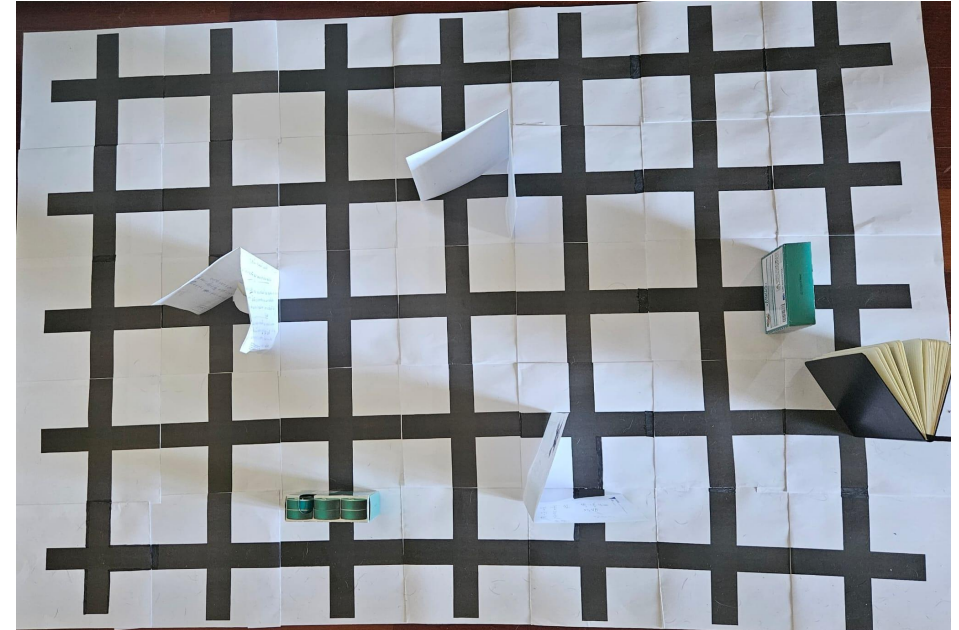
→ **40 mm line width**

→ **20 cm cell span**

The maze can be reconfigured every run by changing the position of the obstacles following some restrictions:

→ Could not be of **dark opaque** texture

→ Had to be placed at the **border between cells**



Real 5x7 maze

Before deployment, parameters were empirically tuned in a **calibration maze** to compensate for differences between **robots and battery levels**.

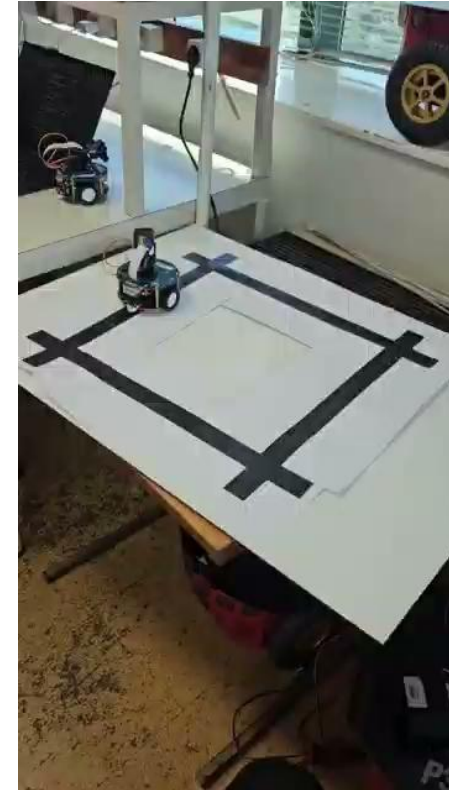
→ Velocity Variables (Linear and Angular)

→ PD Controller Constants

→ Junction Cooldowns Periods

The **longer straight** sections enabled tuning of the **PD controller constants**.

After completing **5 successful laps**, the robot advanced to the real maze.



Calibration Maze

The Dynamic mode was tested throughout 20 different mazes configurations

→ **Obstacles Positions**

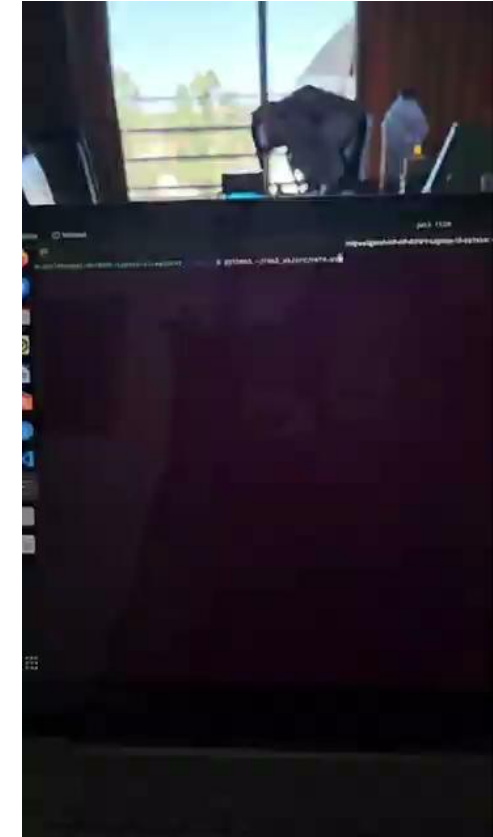
→ **Initial Position and Heading**

→ **Goal Position**

Success Rate of 90% (18/20)

→ **Failure 1: Full Line Loss from PD controller**

→ **Failure 2: Sensor miss read at junction**



Example run of Dynamic Mode

References

- [1] A. Rao, “A guided tour of chapter 4: Markov decision process and bellman equations,” CME 241: Foundations of Reinforcement Learning with Applications in Finance, Lecture Slides, 2023, accessed: 2026-06-04. [Online]. Available: https://web.stanford.edu/class/cme241/lecture_slides/Tour-MDP.pdf
- [2] D. P. Bertsekas, Dynamic Programming and Optimal Control, 4th ed. Belmont, MA: Athena Scientific, 2017, vol. 1.
- [3] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018, chapter 6, Section 6.5, page 157 (Q-Learning: Off-Policy TD Control). [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [4] Mik3Rizzo, “Ros2 alphabot2 package,” GitHub repository, 2022. [Online]. Available: <https://github.com/Mik3Rizzo/alphabot2-ros2>
- [5] Waveshare, “Alphabot2,” Waveshare Wiki, accessed: 2025-06-04. [Online]. Available: <https://www.waveshare.com/wiki/AlphaBot2>
- [6] D. M. Gaspar, M. V. Carneiro, F. P. Valério, and P. G. Mónico, “ist-autonomous-systems-project,” GitHub repository, 2025. [Online]. Available: <https://github.com/davidmarafuzgaspar/ist-autonomous-systems-project>

Thank You!